Day 1: Project Overview

Main problems: Overstock & Stockout.
Solution: Predict future demand and recommend stock quantity.
Technologies:
Frontend: React.js, HTML, CSS, JavaScript
Backend: Python (Flask/Django)
Database: MySQL
Dataset: Hugging Face
f-strings (Formatted Strings): f-strings allow embedding dynamic data within print statements.
Example:

```
x = 69
print(f"The value is {x}")
```

Python Libraries:
- NumPy: Numerical operations
- Pandas: Data manipulation and analysis
- Matplotlib: Data visualization

Day 2: API & Tools

Postman: Used to test APIs by sending requests and viewing responses.
HTTP Meths:
- GET: retrieve a record
- POST: create record
- PUT: update a record
- DELETE: delete a record

APIs: It enables different applications to communicate and exchange data.
Flask: Lightweight Python framework for APIs and web apps.
FastAPI: High-performance framework using Python type hints.
Payload: Actual data sent in an API request.
Endpoint: Specific API URL where a request is sent.
NumPy: Fast mathematical operations.
Pandas: Data manipulation using DataFrames.

Day 3: Git & Virtual Environment

Git Commands:
- git add . : Add all files
- git commit -m "msg" : Commit changes
- git push : Push ce
- git branch : Show current branch
- git fetch : Fetch latest changes
- git fetch --all : Fetch all commits
- git pull

- git checkout branch
- git checkout -b newbranch

Python Environment:
python -m venv envname
env\Scripts\activate
pip install -r requirements.txt

Day 4: Database Concepts

Database: System to store, organize, and manage data.
Relational: MySQL, PostgreSQL – tables with relationships.
Non-relational: MongB – JSON-like documents.
CRUD: Create, Read/Rename, Update, Delete
Normalization:
1NF: Remove repeating values
2NF: Remove partial dependency
3NF: Remove transitive dependency
Primary Key: Uniquely identifies a row.
Foreign Key: Refers to primary key of another table.
Transaction Data: Dynamic data representing daily activities.
MongB: A NoSQL DB storing data in documents.

Day 6: AI & ML Concepts

Artificial Intelligence: Builds machines that can think, learn, and make decisions.
AI Mels: Used for prediction, classification, text generation, etc.
API for AI Mels:
Load/Train mel → API receives input → Mel predicts → API returns result.
LLM (Large Language Mel): Neural network trained on huge text data.
PostgreSQL: Advanced relational DB used for complex queries.
MySQL vs PostgreSQL:
MySQL: Simple and fast.
PostgreSQL: Advanced and used for enterprise.
Vector Database: Stores vector embeddings for similarity search.
Embeddings: Vector representation of text, images, audio.
Machine Learning:
Supervised: labelled data.
Unsupervised: unlabelled data.
Reinforcement Learning: learns through rewards.
Clustering: Groups similar data points without labels.

Day 7 adding repo

learn patterns, make decisions or prediction act autonomously to set some taks without human, but thinks like the human intelligenece
Key Subfields: ML, DL, NLP, RL

Demand forecasting
Dynamic priceing
Collect data
Prepare & engineer features
Train a mel
Evaluate on holdout data using metrics
Deploy to pruction and monitor
Demand forecasting
Demand pricing
customer analytics

1. goal Understanding
2. Planning
3. Taking Actions
4. feedback loop

Agent:

1. check stock levels automatically.
2. Predict when items will be out-of-stock
3. send alert to manager
4. auto-generate purchase orders
5. update demand forecasts daily

types of agent

- reactive agent
- goal-driven agent
- multi-agent

how ai agents plan, think and act
git is the version control
github is cloud hosting for git repositories
onlin platform to host the git
git you should check versions
git -version
git will always tracking ce changes
tracking who made changes in Repository
git config -global user.name "shakthi"
git config -global user.email"shakthi@gmail.com"

it will convey the user message to the assistant and get the output from the assistant and give to the user
user = UserProxyAgent()

assistant = AssistantAgent(name="cer", ce_execution=True)
userProxyAgent will have the controls of the Workflow

Agent with tools
tools are the functions which has some of the main logics
agent will tkae the decision to selcet the tools
in autogen we have userproxyand assitance agent
Assistant Agent is the Smart Al Worker Agent
LLM call will be happening
solving the task are done by the Assistant
can write ce, plan steps, search information
userProxyAgent = Human agent inside the system
it will convey the user message to the assistant and get the output from the assistant and give to the user

LSTM - Special type of RNN
Remember long-term information
Avoid forgetting important patterns
Work well with sequentail/time-serier data
Three gate
Forget Gate
Input Gate
Output Gate
NLP
Sentiment Analysis
Tokenization
smartStock improves inventory decision
stemming
playing - play
Lemmatization
better -> better
better - > go
4. Stop-word removal
5. Text Preprocessing
6. Part-of-Speech (POS)
POS Tagging
7. Named Entity Recognition
dmart
india
PERSON, ORG, GPE, DATE, Pruct
8. building Small NLP Mels

ML technic
supervised learning
classification, Regression and outlier detection
distance between the separating line and the closest data points from each class.
closest points == Support Vectors
Hard Margin Vs Soft Margin
Perfect separation
No misclassification
data clear clean
there should not be noise
perfectly separable
Soft Margin
C paramenter
TF-IDF
term Frequency – Inverse Document Frequency
TF = How often a word appears in a document
IDF = How rare the word is across all the document
aman == word appears 30 times in the doucment and th total word is
1000
kumar == word appears 900 times in the document and the total word in the document is
1000
30/1000 -- > high IDF
900/1000 -- > low IDF
RL = Reinforcemet Learning
Agentic Al
Al Agents
Agent
two main concepts:

- Human in loop
- Without human interaction

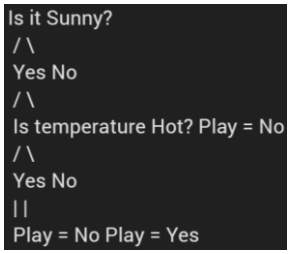
autogen
framework
decision making agent
we have two tools: calculator, unit converter
what is 2+2
Logistic Regression - Classification
0/1, yes/no, spam/not spam
z=w1x1+w2x2+ ... +b
p=o(z)=1+e-z1
if p > 0.5 - class 1

else -> class 0

use of logistic regress

1. fraud detection
2. medical diagnosis
3. customer churn
4. email spam detection

from sklearn.linear_mel import LogisticRegression
mel = LogisticRegression()
Decision Tree



from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier()
Random Forest
Final output = majority vote (classification) or average (regression).
from sklearn.ensemble import
RandomForestClassifier
rf = RandomForestClassifier()
KNN - K- Nearest Neighbors
d=(x1-x2)2+(y1-y2)2
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
K-Means Clustering

k clusters

- . Choose K centroids
- . Assign points to nearest
- centroid
- . Recalculate centroid
- . Repeat

from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3)
Linear Regression
y=mx+c
from sklearn.linear_mel import LinearRegression
XGBoost
Gradient Boosting builds mels sequentially
from xgboost import XGBClassifier
mel = XGBClassifier()

SQL Commands:

1. Select: extracts data from a database
2. Update: Updates data in a database
3. Delete: deletes data from a database
4. insert into: inserts new data into a database
5. create: creates a new database
6. alter: mifies a db
7. drop:
8. truncate

- SELECT column names FROM tablename;
- SELECT DISTINCT column name FROM table name;
- SELECT COUNT (DISTINCT Country) FROM table name;
- **WHERE Clause:** SELECT * FROM table name WHERE column name = value of the column;

<> (or) !=

operator.

1. =
2. >
3. <
4. >=
5. <=
6. Between
7. LIKE
8. IN

name like %s

- salary between 10000 and 50000
- SELECT * FROM pructs WHERE price BETWEEN 500 AND 1500;

WHERE:

1. AND
2. OR
3. NOT

- select * from employees where salary>50000 AND department =cse
- SELECT * FROM Employees WHERE country = 'India' AND salary >30000;
- SELECT * FROM employees WHERE dept = 'IT' AND salary > 60000 OR dept = 'HR';
- SELECT * FROM employees WHERE (dept='HR' OR dept='Sales') AND salary < 50000;
- Select * FROM Customers WHERE country='Germany' AND (city='Berlin' OR city = 'stuttgart');
- SELECT * FROM Employees WHERE Department = 'IT' AND (Salary > 60000 OR Name = 'rammu');
- SELECT * FROM employees where salary>50000 AND (department = finance OR development)

- SELECT * FROM employees WHERE dept = 'IT' OR (dept = 'HR' AND salary >50000);

Order by

- SELECT * FROM STUDENTS ORDER BY id

ASC DEsC Ascending order by default

- SELECT * FROM Students ORDER BY Course ASC
- SELECT * FROM students ORDER BY marks ASC;
- SELECT pruct_name, price FROM pructs ORDER BY price ASC;
- SELECT * FROM Students ORDER BY Course ASC, Marks DESC;
- SELECT * FROM Employees ORDER BY department ASC, name ASC;

INSERT INTO

- insert into employees (id,name) VALUES(1,Siva)
- INSERT INTO Table name(column name1, column name 2,)values('value 1", value2');

Update;

- UPDATE students SET marks = 95 WHERE id = 10;
- Update students SET name = 'shakthi', age = '23' WHERE id=06;
- UPDATE employees SET salary = 50000 WHERE id = 3;
- UPDATE students SET name = 'Sneha R', age = 20 WHERE id = 3;

Delete

- DELETE FROM students WHERE id = 2;

Aggregated function

- MIN
- MAX
- COUNT
- AVG
- SUM

Like Operator.

1. a%
2. %a
3. %or%
4. -r%
5. a_%
6. a_%
7. a%0

- select fname from employee where fname like 'a_%';
- SELECT * FROM words WHERE name LIKE 'a__%';
- SELECT * FROM words WHERE name LIKE 'a%0';
- SELECT * FROM Employees WHERE Name LIKE 'a_%';

IN, NOT IN

- SELECT * FROM employees WHERE department IN ('HR",Finance');
- SELECT * FROM custormers where country IN ('Germany', 'France",UK')

- SELECT * FROM students WHERE city IN ('Mumbai', 'Delhi');

Joins
table 1: std id, std name, address
table 2: subject id, std id, subject name
inner join result will be the std id

String Methods

1. strip() – Removes leading and trailing spaces.
" hello ".strip() # 'hello'
2. split() – Splits a string into a list by separator.
"a,b,c".split(",") # ['a','b','c']
3. join() – Joins elements of a list into a string.
"-".join(['a','b']) # 'a-b'
4. replace() – Replaces part of a string with another text.
"hello world".replace("world", "Python") # 'hello Python'
5. upper() – Converts to uppercase.
"hello".upper() # 'HELLO'
6. lower() – Converts to lowercase.
"HELLO".lower() # 'hello'
7. startswith() – Checks if the string starts with given text.
"python".startswith("py") # True
8. endswith() – Checks if the string ends with given text.
"python".endswith("on") # True
9. find() – Returns index of substring (–1 if not found).
"hello".find("l") # 2
10. isdigit() – Checks if string contains only digits.
"123".isdigit() # True
11. isalpha() – Checks if string contains only alphabets.
"abc".isalpha() # True

List Methods

1. append() – Adds an element at the end.
lst = [1]; lst.append(2) # [1,2]
2. extend() – Adds multiple elements.
lst = [1]; lst.extend([2,3]) # [1,2,3]
3. insert() – Inserts at a specific index.
lst = [1,3]; lst.insert(1,2) # [1,2,3]
4. remove() – Removes first matching element.
lst = [1,2,2]; lst.remove(2) # [1,2]
5. pop() – Removes and returns element at index.
lst = [1,2]; lst.pop(1) # returns 2
6. index() – Returns index of first occurrence.

[10,20,30].index(20) # 1

7. count() – Counts occurrences of value.

[1,2,2,3].count(2) # 2

8. sort() – Sorts the list.

lst = [3,1,2]; lst.sort() # [1,2,3]

9. reverse() – Reverses list order.

lst = [1,2,3]; lst.reverse() # [3,2,1]

Dictionary Methods

1. get() – Returns value for a key (or default).

{"a":1}.get("a") # 1

2. keys() – Returns all keys.

{"a":1}.keys() # dict_keys(['a'])

3. values() – Returns all values.

{"a":1}.values() # dict_values([1])

4. items() – Returns key-value pairs.

{"a":1}.items() # dict_items([('a',1)])

5. update() – Updates dictionary with another.

d={"a":1}; d.update({"b":2}) # {'a':1,'b':2}

6. pop() – Removes a key and returns its value.

{"a":1}.pop("a") # 1

7. popitem() – Removes and returns last inserted item.

{"a":1,"b":2}.popitem() # ('b',2)

8. clear() – Removes all items.

d={"a":1}; d.clear() # {}

Set Methods

1. add() – Adds an element.

s={1}; s.add(2) # {1,2}

2. remove() – Removes element; error if missing.

s={1,2}; s.remove(2) # {1}

3. discard() – Removes element if present (no error).

s={1,2}; s.discard(3) # {1,2}

4. pop() – Removes and returns a random element.

s={1,2,3}; s.pop()

5. clear() – Removes all elements.

s={1,2}; s.clear() # set()

6. union() – Returns union of sets.

{1,2}.union({2,3}) # {1,2,3}

7. intersection() – Returns common elements.

{1,2,3}.intersection({2,3,4}) # {2,3}

8. difference() – Elements in first set but not second.

{1,2,3}.difference({2}) # {1,3}

File I/O Methods

1. open() – Opens a file for reading/writing.

f = open("data.txt", "r")

2. read() – Reads entire file.

open("a.txt").read()

3. readline() – Reads single line.

open("a.txt").readline()

4. readlines() – Reads all lines as a list.

open("a.txt").readlines()

5. write() – Writes text to file.

f=open("a.txt","w"); f.write("hello")

6. writelines() – Writes list of lines.

f=open("a.txt","w"); f.writelines(["a\n","b\n"])

7. close() – Closes file.

f.close()

General-Purpose Functions

1. len() – Returns length of an object.

len([1,2,3]) # 3

2. range() – Generates a sequence of numbers.

list(range(3)) # [0,1,2]

3. print() – Prints output.

print("Hello")

4. type() – Returns the data type.

type(5) # <class 'int'>

5. id() – Returns memory address of object.

id(10)

6. sorted() – Returns a new sorted list.

sorted([3,1,2]) # [1,2,3]

7. enumerate() – Returns index + value pairs.

list(enumerate(['a','b'])) # [(0,'a'),(1,'b')]

8. zip() – Combines multiple iterables element-wise.

list(zip([1,2],[3,4])) # [(1,3),(2,4)]

Miscellaneous:

globals()
locals()
collable()
exec()
eval()

Exception Handling:
try
except
finally

a = input("Enter.")
format()

class and object related:
getattr()
setattr()
hasattr()
delattr()
isinstance()
issubclass()

add_ten = lambda x:x+10
print(add_ten(5))
def (a,b)
a = lambda a,b : a*b
print(a(3,4))

1. What are Python's key feature?
   Interpreted, high-level, dynamically typed, object-oriented, cross-platform, supports multiple paradigms, and has extensive libraries.
2. What are python's data types?
   int, float, complex, list, str, tuple, set, dict, bool, nonetype
3. what isPEP 8 and why is it important?
   python's style guide, ensre consistent, clean and readable code.
4. delete file?
   os.remove
5. monkey patching?
   dynamically changing class/module behavious at runtime
6. syntax vs runtime error
   invalid code error and runtime occurs during execution
7. whare python's memort management features?
   Private heap, reference counting, garbage collector
8. How is python interpreted?
   complied into bytecode
9. explain namespaces.
   mapping of variable names to objects
10. how to merge dictionaries?

dict1.update(dict2).

11. what are closures?

function with access to variables fromenclosing scope.

12. What is a lambda function?

A **lambda function** is a small **anonymous function** in Python defined using the lambda keyword and can have multiple arguments but **only one expression**.

13. What is pip?

pip (Python Installer Package) is a tool used to **install, update, and manage Python libraries and packages**.

14. Multiple vs Multilevel Inheritance

Multiple inheritance occurs when a class inherits from **more than one parent class**.

Multilevel inheritance occurs when a class inherits from a class which is already derived from another class.

15. Explain mutable vs immutable objects

Mutable objects can be changed after creation (e.g., list, set, dictionary).

Immutable objects cannot be changed after creation (e.g., int, float, string, tuple).

16. What are Python's built-in data structures?

Python's built-in data structures include **List, Tuple, Set, and Dictionary**, which are used to store and organize data.

17. Explain indentation in Python

Indentation in Python is used to define **blocks of code**. It is mandatory and replaces braces {} used in other languages.

18. List vs Tuple differences

A **list** is mutable and can be modified after creation.

A **tuple** is immutable and cannot be modified after creation.

19. How are sets used?

Sets are used to store **unique elements**, remove duplicates, and perform mathematical operations like **union, intersection, and difference**.

20. What are dictionaries?

Dictionaries are data structures that store data in **key–value pairs**, allowing fast data retrieval using keys.

21. Shallow vs Deep Copy

A **shallow copy** copies object references, not nested objects.

A **deep copy** creates a completely independent copy of all objects, including nested ones.

22. Function vs Method

A **function** is a block of reusable code defined independently.

A **method** is a function that is **associated with an object or class**.

23. What is *args and **kwargs?

*args allows a function to accept a **variable number of positional arguments**.

kwargs allows a function to accept a **variable number of keyword arguments.

24. What are decorators?

Decorators are functions used to **modify or extend the behavior of another function** without changing its code.
Memory and object
Management:
del()
gc.collect()
Exception Handling:
try
except
finally
we keep the code that may rise exception inside "try" block if any exception raises "except" block will run, it handles the exception finally block will run in both situations
import gc
Working with iterables:
Next()
Iter()
Decorators and Metaprogramming
staticmethod()
classmethod()

1. What is functional programming in Java?
   Functional programming in Java is a programming style that uses **lambda expressions, functional interfaces, and streams** to write concise, declarative, and immutable code.
2. What is JDK?
   JDK (Java Development Kit) is a software package that provides tools to **develop, compile, and run Java programs**, including JRE and development tools like javac.
3. What is the == operator in Java?
   The == operator is used to **compare references** of objects and **values of primitive data types**.
4. What is .equals() in Java?
   The .equals() method is used to **compare the content or values of two objects**.
5. What is a private constructor?
   A private constructor restricts object creation outside the class and is mainly used in **Singleton classes**.
6. What is encapsulation?
   Encapsulation is the process of **wrapping data and methods into a single unit (class)** and restricting direct access using access modifiers.

7. What is abstraction?
   Abstraction is the process of **hiding implementation details** and showing only essential features using **abstract classes or interfaces**.
8. What is inheritance?
   Inheritance allows a class to **acquire properties and behaviors of another class** using the extends keyword.
9. What are primitive data types in Java?
   Primitive data types are **basic data types** that store simple values, such as int, float, char, boolean, etc.
10. What is type casting?
    Type casting is the process of **converting one data type into another**, either implicitly or explicitly.
11. What is static in Java?
    Static members belong to the **class rather than objects** and can be accessed without creating an object.
12. What is non-static in Java?
    Non-static members belong to **objects** and require object creation to access them.
13. What is the this keyword?
    this refers to the **current object of the class**.
14. What is the super keyword?
    super is used to **access parent class variables, methods, or constructors**.
15. What is final in Java?
    The final keyword prevents **modification**, inheritance, or method overriding.
16. What is the finally block?
    The finally block executes **regardless of exception occurrence**, mainly used for cleanup code.
17. What is finalize()?
    finalize() is a method called by the **Garbage Collector before object destruction**.
18. What is String in Java?
    String is an **immutable object** that represents a sequence of characters.
19. What is StringBuffer?
    StringBuffer is a **mutable and thread-safe** class used to modify strings.
20. What is StringBuilder?
    StringBuilder is a **mutable but non-thread-safe** class used for faster string operations.
21. What is the Collection framework?
    The Collection framework provides **classes and interfaces** to store and manipulate groups of objects.
22. What is List?
    List is an **ordered collection** that allows duplicate elements.
23. What is Set?
    Set is a collection that **does not allow duplicate elements**.

24. What is throw?
throw is used to **explicitly throw an exception**.
25. What is throws?
throws is used to **declare exceptions** that may occur in a method.
26. What is deadlock?
Deadlock is a situation where **two or more threads wait indefinitely** for resources held by each other.
27. How can deadlock be prevented?
Deadlock can be prevented by **proper resource ordering, avoiding nested locks, and using timeout mechanisms**.
28. What is a lambda function?
A lambda function is an **anonymous function** used to implement functional interfaces.
29. What is map() in Java streams?
map() is used to **transform each element** of a stream.
30. What is flatMap()?
flatMap() converts **multiple streams into a single stream**.
31. Does an interface have a constructor?
No, an interface **does not have a constructor** because it cannot be instantiated.
32. What is multiple inheritance using interfaces?
Java supports multiple inheritance by allowing a class to **implement multiple interfaces**.
33. What is the diamond problem?
The diamond problem occurs when **multiple interfaces have default methods with the same signature**.
34. What is method hiding?
Method hiding occurs when a **static method in a subclass has the same signature** as a static method in the parent class.
35. What is a static block?
A static block is used to **initialize static variables** and executes before the main method.
36. What is a constructor?
A constructor is a **special method used to initialize objects** when they are created.

15/12/2025

1. Functional Consultant / Business Analyst - Designs the **Functional Specification Document (FSD)**

A Functional Consultant or Business Analyst **understands business requirements**, interacts with stakeholders, and converts business needs into a **functional specification document** that explains *what the system should do*.

2. Developer / Programmer - Creates the **Technical Design Document (TDD)**

A Developer or Programmer **writes code**, designs system architecture, and converts functional specifications into a **technical solution** by developing applications, APIs, and databases.

3. Testing / QA (Quality Assurance) - Prepares **Test Scripts and Test Cases**
The QA team **tests the application** based on the functional specification to ensure the software works as expected, finds bugs, and verifies quality before release.

4. Administration - Manages **Application, Database, and Network**
Administration ensures **system availability and performance** by managing servers, databases, applications, backups, deployments, and network infrastructure.

5. Security Team - Ensures **Data and System Security**
The Security team **protects applications and data** from threats by implementing authentication, authorization, encryption, firewalls, and security policies.

6. Product / Project Management - Planning and Delivery
Product or Project Managers **plan, track, and control projects**, manage timelines, resources, budgets, and ensure the product meets business goals and is delivered on time.

7. Sales & Marketing - Product Promotion and Revenue
Sales & Marketing teams **promote the product**, identify customers, generate leads, explain product value, and drive revenue growth.

8. Human Resource (HR) - Workforce Management
HR manages **recruitment, employee relations, payroll, performance evaluation, training, and compliance** within the organization.

9. Documentation Team - User & Technical Documentation
The Documentation team **creates user manuals, system guides, and help documents** to explain how to use and maintain the application.

10. Support Team - Post-deployment Assistance
The Support team **assists users after deployment**, resolves issues, provides troubleshooting, and ensures smooth system operation.

Agile and waterfall

A * Search Algorithm

Start Node: A Goal Node: F Nodes: A, B, C, D, E, F Edges with weights: (A, B, 1), (A, C, 4), (B, D, 3), (B, E, 5), (C, F, 2), (D, F, 1), (D, E, 1), (E, F, 2)
Sorting algorithm arrange in some order
In axx or des

Bubble sort

11 12 22 25 64

64 25 12 22 11
64 12 25 22 11
64 12 25 11 22
12 64 25 11 22
12 25 64 11 22
12 25 11 64 22
12 25 11 22 64
12 11 25 22 64
12 11 22 25 64
11 12 22 25 64

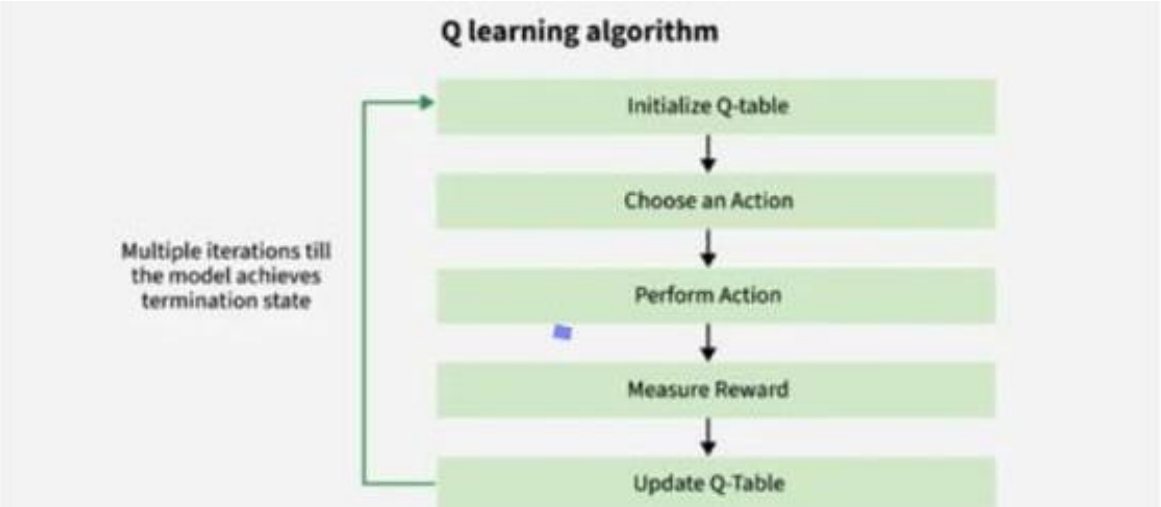
Reinforcement Learning: q-learning-value-based learning for decision-making

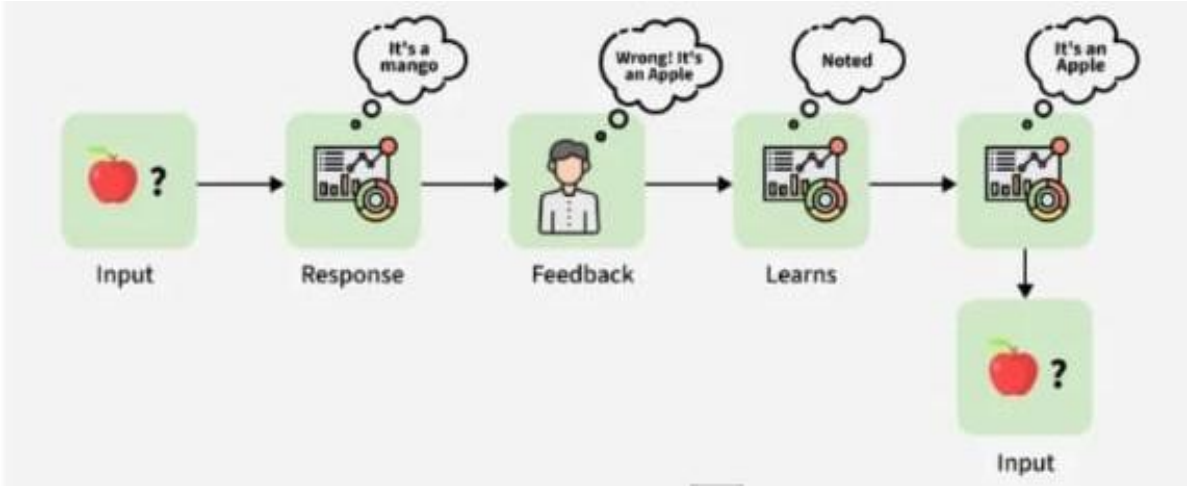
Deep q-networks(DON) – combines q-learning with deep neural networks policy gradient methods – directly optimize policies

Input(apple) -> response (mongo) -> feedback (wrong answer the correct one will be apple) -> learns(note) model(rewrite it as the apple) takes this as the imput

Q- Values or Action Values. Reward and Episodes Temporal Difference or TD-Update
← Q(S,A) - Q(S,A) +a(R+yQ(S',A') -Q(S,A))

Greedy policy exploitation exploration





Methods for Determining Q-values
1. Temporal difference (TD) - calculate the comparing the current state and action values with the previous onces.

Bellman's Equation: Q(s,a)=R(s,a) +ymaxaQ(s',a)

1. Breadth First Search (BFS)

BFS is a search algorithm that explores all nodes level by level from the starting node before moving deeper. It uses a **queue** and always finds the **shortest path** in an unweighted graph.

2. Depth First Search (DFS)

DFS is a search algorithm that explores as deep as possible along one branch before backtracking. It uses a **stack or recursion** and requires less memory than BFS.

3. Depth Limited Search (DLS)

Depth Limited Search is a variation of DFS that limits the depth of search to a predefined level, preventing infinite exploration.

4. Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the **lowest path cost** first. It guarantees the optimal solution when costs are positive.

5. Traveling Salesman Problem (TSP)

The Traveling Salesman Problem is an optimization problem where a salesman must visit all cities exactly once and return to the starting city with minimum travel cost.

6. Game Playing (Minimax / Game Model – “Gman”)

Game playing in AI uses algorithms like **Minimax** to determine optimal moves by assuming the opponent also plays optimally.

7. Face Search (Face Recognition/Search)

Face search is a computer vision technique used to identify or verify a person by comparing facial features with stored images.

8. Principal Component Analysis (PCA)

PCA is a dimensionality reduction technique that converts high-dimensional data into fewer important components while preserving maximum variance.

9. VGG-16

VGG-16 is a deep **Convolutional Neural Network (CNN)** with 16 layers used for image classification tasks.

10. Convolutional Neural Network (CNN)

CNN is a deep learning model mainly used for image and video processing by automatically extracting spatial features.

11. Recurrent Neural Network (RNN)

RNN is a neural network designed to process **sequential data** by using feedback connections.

12. Long Short-Term Memory (LSTM)

LSTM is a special type of RNN that solves the **vanishing gradient problem** and can learn long-term dependencies in sequence data.

13. Bidirectional Search

Bidirectional search runs two searches simultaneously—one from the start and one from the goal—to reduce search time.

14. Hill Climbing Algorithm

Hill climbing is a local search algorithm that continuously moves toward a better solution based on a heuristic until no improvement is possible.